On Software Systems, Models, and Missed Opportunities

A note synthesizing observations on software architecture, domain modeling, and the VistA case study.

Every Software System Is a Model

Any software system is a simplified model of a prototype — whether that prototype is a real-world process, an organization, or a conceptual domain. The art of software engineering lies in knowing what to include, what to deliberately exclude, and where exactness matters. A model that attempts to capture everything is as useless as one that captures nothing — both collapse under their own weight.

Two recurring failure modes undermine this ideal. The first is the absence of a model entirely — the developer receives requirements and immediately thinks in technology: tables, classes, APIs, frameworks. The real-world domain is bolted on afterward, producing a system that technically works but fights its own purpose at every turn. The second failure mode is more insidious: the model is forced to fit the available technology stack rather than the prototype it should represent. Familiar patterns are imposed on unfamiliar tools. The result in both cases is the same — constant rewriting of systems that are in use, because the foundation never matched the reality it was meant to serve.

Users working around a system are the most reliable signal that the model has diverged from the prototype. That divergence, left unaddressed, compounds with every passing year.

MUMPS and the Persistence Opportunity

MUMPS (Massachusetts General Hospital Utility Multi-Programming System, 1966) offered something genuinely rare: persistence as a first-class feature of the language itself. Not a library, not a framework, not an ORM — the language natively stored data. A variable prefixed with ^ simply persisted across sessions, without serialization, without connection management, without schema migration. The infrastructure disappeared into the language, leaving only the domain model to be expressed.

This was an architectural gift. The correct response would have been to build a clean, presentation-independent domain model at the server level — one that clinical data could inhabit naturally, and from which any presentation layer (terminal menus, graphical windows, web interfaces, voice, images) could be served without touching the model itself. The model would exist once. The presentation would be arbitrary and replaceable.

The healthcare domain further invited something more sophisticated: multiple correlated models of the same clinical reality. A clinical model, a billing model, an administrative model, a pharmacological model — each a different perspective on the same underlying events, each valid, each necessary, each maintainable independently while remaining correlated through explicit architectural relationships. MUMPS globals, with their hierarchical and sparse nature, were structurally suited to exactly this vision.

What VistA Built Instead

VistA (Veterans Information Systems and Technology Architecture) grew as a collection of independent packages built by teams with different goals and no shared architectural governance. The decentralization was intentional and productive in the early years — local clinical staff knew local needs, MUMPS made local development accessible, and fast iteration on real problems was possible without bureaucratic bottleneck. What worked as a local innovation engine failed as a coherent system.

The persistence opportunity was not taken. Instead of a clean domain model, VistA built terminal presentation logic directly into MUMPS code — mixing business rules with control sequences, replicating the same entanglement between UI and model that plagues every generation of application development. FileMan, the data management layer, imposed relational-style concepts — schemas, data dictionaries, field definitions — onto a hierarchical store that did not need them. The tool was used against its own design intent.

There was never the organizational will to rethink the architecture. Leadership turned over. Clinical risk aversion made change dangerous. Tribal knowledge retired with its holders. Each modernization initiative — HealtheVet, VistA Evolution — attracted feature additions rather than architectural investment. The chaos was never controlled, only accumulated.

The Reliability Paradox

Despite architectural disorder at the application layer, the VistA database proved remarkably reliable across decades, hardware generations, and management cycles. This was not accidental. The GT.M engine (now YottaDB) provided ACID guarantees at the global node level, built-in journaling, and optimistic concurrency — originally engineered for financial transaction processing, where data loss was measured in dollars before it was measured in patient outcomes. FileMan, for all its architectural impurity, served as a consistent gatekeeper: nothing entered the database without passing through its validation layer.

The lesson is precise: a reliable system does not require elegant architecture throughout. It requires a solid foundation that does not lie, consistent enforcement at critical data boundaries, and failure modes that are understood and handled. VistA had chaos at the application layer and discipline at the data layer. The data was always trustworthy even when the code accessing it was not elegant. The foundation quality outlasted every architectural mistake built upon it.

The General Principle

Every tool embeds a model of its intended prototype. Using a tool well means understanding what prototype it was designed to serve, and aligning its use to that embedded model. Using a tool poorly means ignoring that model and imposing a more familiar one — fighting the tool, receiving a fraction of its value, and building the same problems into a new technology stack.

The deepest software projects fail not because of bad code, wrong frameworks, or inadequate testing — but because the system never accurately modeled the prototype it was meant to serve. Rewriting such systems in modern technology faithfully reproduces the original confusion. The rewriting cycle continues not because the technology was wrong, but because the model was never right.

The tragedy of VistA is specific: the foundation existed to build it correctly. MUMPS persistence, GT.M reliability, and the hierarchical global structure were perfectly matched to the clinical data prototype. The insight needed to use that foundation well was never present at the right moment, in the right place, with the right authority to act on it. What remains is a cautionary record of what becomes possible — and what is lost — when tools and prototypes align by accident but architecture is never deliberately chosen.

Observations distilled from conversation — March 2026